

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра автоматики и процессов управления

ОТЧЕТ
по лабораторной работе №6
по дисциплине «Программирование»
Тема: Вычисление значений кусочно-заданной функции

Студент гр. 6391

Рудой И.В.

Преподаватель

Писарев А.С.

Санкт-Петербург

2026

Цель работы (задание)

Вычислить и вывести на экран в виде таблицы значения функции F на интервале от $X_{\text{нач.}}$ до $X_{\text{кон.}}$ с шагом dX .

$$F = \begin{cases} ax^2 + b, & \text{при } x < 0 \text{ и } b \neq 0 \\ \frac{x-a}{x-c}, & \text{при } x > 0 \text{ и } b = 0 \\ \frac{x}{c}, & \text{в остальных случаях} \end{cases}$$

где a, b, c – действительные числа.

Функция F должна принимать действительное значение, если выражение

$$(A_{\text{ц}} \text{ ИЛИ } B_{\text{ц}}) \text{ И } (A_{\text{ц}} \text{ ИЛИ } C_{\text{ц}})$$

не равно нулю, и целое значение – в противном случае. Через $A_{\text{ц}}, B_{\text{ц}}, C_{\text{ц}}$ обозначены целые части значений a, b, c ; операции И и ИЛИ – поразрядные. Значения $a, b, c, X_{\text{нач.}}, X_{\text{кон.}}, dX$ ввести с клавиатуры.

Алгоритм решения

Программа выполняет следующие шаги:

- 1. Ввод данных.** С клавиатуры вводятся коэффициенты a, b, c и параметры интервала: $X_{\text{нач.}}, X_{\text{кон.}}, dX$.
- 2. Проверка корректности ввода.** Если $dX = 0$ – выводится ошибка (иначе цикл заиклится). Если знак шага не совпадает с направлением интервала ($X_{\text{нач.}} > X_{\text{кон.}}$ при $dX > 0$, либо $X_{\text{нач.}} < X_{\text{кон.}}$ при $dX < 0$) – выводится ошибка и программа завершается.
- 3. Вычисление целых частей коэффициентов.** $A_{\text{ц}} = \text{int}(a)$, $B_{\text{ц}} = \text{int}(b)$, $C_{\text{ц}} = \text{int}(c)$. Они нужны для проверки условия вывода: $((A_{\text{ц}} \mid B_{\text{ц}}) \& (A_{\text{ц}} \mid C_{\text{ц}})) \neq 0$.
- 4. Выбор направления обхода.** В зависимости от знака шага выбирается один из двух циклов:
 - при $dX > 0$: for (double $x = X_{\text{н.}}$; $x \leq X_{\text{к.}}$; $x += dX$) – x растёт от $X_{\text{нач.}}$ до $X_{\text{кон.}}$;

- при $dX < 0$: for (double $x = X_n$; $x \geq X_k$; $x += dX$) – x убывает от $X_{нач.}$ до $X_{кон.}$

Такой подход гарантирует корректную работу при любом знаке шага.

5. Вычисление F. Для каждой точки x вызывается функция $F(x, a, b, c, A_c, B_c, C_c)$. В зависимости от условий выбирается одна из трёх ветвей функции:

- если $x < 0$ и $b \neq 0$: $F = a \cdot x^2 + b$;
- если $x > 0$ и $b = 0$: $F = (x - a) / (x - c)$;
- в остальных случаях: $F = x / c$.

Перед делением проверяется, не равен ли знаменатель нулю. Если равен, то точка помечается как undefined.

6. Вывод значения:

- если функция не определена – выводится undefined;
- если $((A_c | B_c) \& (A_c | C_c)) \neq 0$ – выводится действительное значение F с 4 знаками после запятой;
- иначе – выводится целая часть F (отбрасывание дробной части через $(\text{int})F$).

Строки выводятся с помощью printf: значения x и $F(x)$ выравниваются по ширине поля.

Текст программы

```
#include <iostream>
#include <cstdio>

using namespace std;

//Вычислить и вывести на экран в виде таблицы значения функции F на интервале
//от Xнач.до Xкон.с шагом dX.
//где a, b, c – действительные числа.
//Функция F должна принимать действительное значение, если выражение
//(Aц ИЛИ Bц) И (Aц ИЛИ Cц)
//не равно нулю, и целое значение в противном случае.Через Aц, Bц и Cц
обозначены
//целые части значений a, b, c, операции И и ИЛИ – поразрядные.Значения
//a, b, c, Xнач., Xкон., dX ввести с клавиатуры.

void F(double x, double a, double b, double c, int Ac, int Bc, int Cc)
{
    double f;
```

```

bool defined = true;

if (x < 0 && b != 0) {
    f = a * x * x + b;
}
else if (x > 0 && b == 0) {
    if (x - c == 0) {                // защита от деления на ноль
        defined = false;
    }
    else {
        f = (x - a) / (x - c);
    }
}
else {
    if (c == 0) {                    // защита от деления на ноль
        defined = false;
    }
    else {
        f = x / c;
    }
}

if (!defined) {
    printf("%10.4f %15s\n", x, "undefined");
}
else if (((Ac | Bc) & (Ac | Cc)) != 0) {
    printf("%10.4f %15.4f\n", x, f);    // действительное значение
}
else {
    printf("%10.4f %15d\n", x, (int)f);    // целое значение
}
}

int main()
{
    double a, b, c, Xn, Xk, dX;

    cout << "Введите коэффициенты a, b и c" << endl;
    cin >> a >> b >> c;

    cout << "Введите значения Xнач. до Xкон. и шаг dX" << endl;
    cin >> Xn >> Xk >> dX;

    // Проверка корректности ввода
    if (dX == 0) {
        cout << "Ошибка: шаг dX не может быть равен нулю" << endl;
        return 1;
    }
    if (Xn > Xk && dX > 0) {
        cout << "Ошибка: Xнач больше Xкон при положительном шаге" << endl;
        return 1;
    }
    if (Xn < Xk && dX < 0) {
        cout << "Ошибка: Xнач меньше Xкон при отрицательном шаге" << endl;
    }
}

```

```

        return 1;
    }

    // Целые части коэффициентов для поразрядных операций
    int Ac = int(a);
    int Bc = int(b);
    int Cc = int(c);

    // Заголовок таблицы
    printf("%10s %15s\n", "x", "F(x)");
    printf("-----\n");

    // Выбор направления обхода в зависимости от знака шага
    if (dX > 0) {
        // Положительный шаг: x растёт от Xn до Xk
        for (double x = Xn; x <= Xk; x += dX) {
            F(x, a, b, c, Ac, Bc, Cc);
        }
    }
    else {
        // Отрицательный шаг: x убывает от Xn до Xk
        for (double x = Xn; x >= Xk; x += dX) {
            F(x, a, b, c, Ac, Bc, Cc);
        }
    }

    return 0;
}

```

Описание тестов

Для каждого теста указаны входные данные и ожидаемый результат.

Тест 1. Обычный случай – все три ветки функции

Вход: $a = 2, b = 1, c = 3$; $X_{\text{нач.}} = -2, X_{\text{кон.}} = 2, dX = 1$

Условие: $((2|1) \& (2|3)) = 3 \neq 0 \rightarrow F$ выводится как действительное число.

Ожидаемый вывод:

x	F(x)	
-----	-----	
-2.0000	9.0000	$\leftarrow a \cdot x^2 + b \ (x < 0, b \neq 0)$
-1.0000	3.0000	
0.0000	0.0000	$\leftarrow x/c \ (\text{остальной случай})$
1.0000	0.3333	$\leftarrow x/c \ (b \neq 0, \text{ вторая ветка не берётся})$
2.0000	0.6667	

Пояснение: проверяется корректный выбор ветки в зависимости от x и b .

Тест 2. Обычный случай – ветка $(x-a)/(x-c)$ при $b = 0$

Вход: $a = 1, b = 0, c = 4$; $X_{\text{нач.}} = 1, X_{\text{кон.}} = 5, dX = 1$

Условие: $((1|0) \& (1|4)) = 1 \neq 0 \rightarrow$ действительный формат.

Ожидаемый вывод:

x	F (x)	
1.0000	-0.0000	$\leftarrow (1-1)/(1-4) = -0$
2.0000	-0.5000	$\leftarrow (2-1)/(2-4) = -0.5$
3.0000	-2.0000	$\leftarrow (3-1)/(3-4) = -2$
4.0000	undefined	$\leftarrow$ деление на $(4-4) = 0$
5.0000	4.0000	$\leftarrow (5-1)/(5-4) = 4$

Пояснение: при $b = 0$ и $x > 0$ используется вторая ветка; в точке $x = c$ функция не определена.

Тест 3. Граничный случай – деление на ноль ($c = 0$)

Вход: $a = 1, b = 2, c = 0$; $X_{\text{нач.}} = -1, X_{\text{кон.}} = 1, dX = 1$

Условие: $((1|2) \& (1|0)) = 1 \neq 0 \rightarrow$ действительный формат (где определено).

Ожидаемый вывод:

x	F (x)	
-1.0000	3.0000	$\leftarrow a \cdot x^2 + b$
0.0000	undefined	$\leftarrow x/c, c=0$
1.0000	undefined	$\leftarrow x/c, c=0$

Пояснение: при $c = 0$ в «остальных случаях» (и при $x \geq 0$) происходит деление на ноль $\rightarrow$ undefined.

Тест 4. Граничный случай – условие вывода целых значений

Вход: $a = 0, b = 0, c = 5$; $X_{\text{нач.}} = -2, X_{\text{кон.}} = 2, dX = 1$

Условие: $((0|0) \& (0|5)) = 0 \rightarrow F$ выводится как целое число.

Ожидаемый вывод:

x	F (x)	
-2.0000	0	$\leftarrow x/c = -0.4 \rightarrow (\text{int})-0.4 = 0$
-1.0000	0	$\leftarrow x/c = -0.2 \rightarrow 0$
0.0000	0	
1.0000	0	$\leftarrow x/c = 0.2 \rightarrow 0$
2.0000	0	$\leftarrow x/c = 0.4 \rightarrow 0$

Пояснение: когда поразрядное выражение равно нулю, дробная часть отбрасывается. Здесь все значения по модулю меньше 1, поэтому в таблице нули.

Тест 5. Отрицательный шаг

Вход: $a = 1, b = 2, c = 1$; $X_{\text{нач.}} = 2, X_{\text{кон.}} = -2, dX = -1$

Ожидаемый вывод:

x	F (x)	
2.0000	2.0000	$\leftarrow x/c$

1.0000	1.0000	
0.0000	0.0000	
-1.0000	3.0000	$\leftarrow a \cdot x^2 + b$
-2.0000	6.0000	

Программа корректно обходит интервал «справа налево» за счёт цикла с условием $x \geq X_k$.

Тест 6. Граничный случай – $X_{нач.} = X_{кон.}$ (одна точка)

Вход: $a = 2, b = 3, c = 1$ $X_{нач.} = 5, X_{кон.} = 5, dX = 1$

При $X_{нач.} = X_{кон.}$ цикл for выполняется ровно один раз: $x = 5$, условие $5 \leq 5$ истинно, после чего шаг даёт 6, и цикл завершается.

Ожидаемый вывод:

x	F(x)	
5.0000	5.0000	$\leftarrow x/c = 5/1$

Пояснение: при совпадении границ интервала программа выводит единственную точку и корректно завершается (не за циклируется и не пропускает значение).

Тест 7. Проверка ввода – некорректные параметры интервала

Проверяются три ошибочные ситуации. Программа должна вывести сообщение и завершиться с кодом 1 (таблица не строится).

7.1. Нулевой шаг:

Ввод: $a=1 \ b=1 \ c=1; X_{нач.}=0 \ X_{кон.}=1 \ dX=0$

Вывод: Ошибка: шаг dX не может быть равен нулю

7.2. $X_{нач.} > X_{кон.}$ при положительном шаге:

Ввод: $a=1 \ b=1 \ c=1; X_{нач.}=5 \ X_{кон.}=1 \ dX=1$

Вывод: Ошибка: $X_{нач.}$ больше $X_{кон.}$ при положительном шаге

7.3. $X_{нач.} < X_{кон.}$ при отрицательном шаге:

Ввод: $a=1 \ b=1 \ c=1; X_{нач.}=1 \ X_{кон.}=5 \ dX=-1$

Вывод: Ошибка: $X_{нач.}$ меньше $X_{кон.}$ при отрицательном шаге

Пояснение: без этих проверок при $dX = 0$ программа либо за циклилась бы.

Результат работы

Ниже приведены скриншоты консоли, подтверждающие корректную работу программы на разработанных тестах.

```

Введите коэффициенты a,b и c
2 1 3
Введите значения Xнач. до Xкон. и шаг dX
-2 2 1

```

x	F(x)
-2.0000	9.0000
-1.0000	3.0000
0.0000	0.0000
1.0000	0.3333
2.0000	0.6667

Рисунок 1. Результаты выполнения теста 1

```

Введите коэффициенты a,b и c
1 0 4
Введите значения Xнач. до Xкон. и шаг dX
1 5 1

```

x	F(x)
1.0000	-0.0000
2.0000	-0.5000
3.0000	-2.0000
4.0000	undefined
5.0000	4.0000

Рисунок 2. Результаты выполнения теста 2

```

Введите коэффициенты a,b и c
1 2 0
Введите значения Xнач. до Xкон. и шаг dX
-1 1 1

```

x	F(x)
-1.0000	3.0000
0.0000	undefined
1.0000	undefined

Рисунок 3. Результаты выполнения теста 3

```

Введите коэффициенты a,b и c
0 0 5
Введите значения Xнач. до Xкон. и шаг dX
-2 2 1

```

x	F(x)
-2.0000	0
-1.0000	0
0.0000	0
1.0000	0
2.0000	0

Рисунок 4. Результаты выполнения теста 4

```

Введите коэффициенты a,b и c
1 2 1
Введите значения Xнач. до Xкон. и шаг dX
2 -2 -1

```

x	F(x)
2.0000	2.0000
1.0000	1.0000
0.0000	0.0000
-1.0000	3.0000
-2.0000	6.0000

Рисунок 5. Результаты выполнения теста 5

```

Введите коэффициенты a,b и c
2 3 1
Введите значения Xнач. до Xкон. и шаг dX
5 5 1

```

x	F(x)
5.0000	5.0000

Рисунок 6. Результаты выполнения теста 6

```

Введите коэффициенты a,b и c
1 1 1
Введите значения Xнач. до Xкон. и шаг dX
0 1 0
Ошибка: шаг dX не может быть равен нулю

```

```

Введите коэффициенты a,b и c
1 1 1
Введите значения Xнач. до Xкон. и шаг dX
5 1 1
Ошибка: Xнач больше Xкон при положительном шаге

```

```

Введите коэффициенты a,b и c
1 1 1
Введите значения Xнач. до Xкон. и шаг dX
1 5 -1
Ошибка: Xнач меньше Xкон при отрицательном шаге

```

Рисунок 7. Результаты выполнения теста 7

Все тесты отработали в соответствии с ожидаемыми результатами. Программа корректно вычисляет значения кусочно-заданной функции, обрабатывает точки с делением на ноль (выводит undefined), правильно выбирает между действительным и целым выводом по побитовому условию и корректно работает с отрицательным шагом и произвольными границами интервала.

Вывод

В ходе лабораторной работы была разработана программа на языке C++, вычисляющая значения кусочно-заданной функции F на заданном интервале с заданным шагом.

Для корректной работы с любым знаком шага обход интервала реализован через конструкцию `if / else` с двумя циклами `for (double x =  $X_n$ ; ...; x += dX)`: при положительном шаге используется условие $x \leq X_k$, при отрицательном – $x \geq X_k$.

Логика вычисления F вынесена в отдельную функцию, что позволило избежать дублирования кода между двумя циклами.

Учтены возможные особые случаи: деление на ноль (точка помечается как `undefined`), некорректный ввод (нулевой шаг, несовпадение направления шага и интервала).

Условие выбора между действительным и целым выводом реализовано через поразрядные операции И и ИЛИ над целыми частями коэффициентов, как указано в задании.

Тестирование подтвердило правильность работы программы на различных наборах данных.

В процессе работы закрепились навыки работы с условными операторами, циклами, вещественной арифметикой, функциями и поразрядными операциями в языке C++.